

Preface

Many compiler books have been published over the years, so why another one? Let me be perfectly clear. This book is designed primarily for use as a textbook in a one-semester course for undergraduate students and beginning graduate students. The only prerequisites for this book are familiarity with basic algorithms and data structures (lists, maps, recursion, etc.), a rudimentary knowledge of computer architecture and assembly language, and some experience with the Java programming language or a closely related language such as Kotlin, C#, or C++. Most undergraduate computer science majors will have covered these topics in their first two years of study. Graduate students who have never had a course in compilers will also find the book useful, especially if they undertake some of the more challenging project exercises described in Appendix B.

A complete study of compilers could easily fill several graduate-level courses, and therefore some simplifications and compromises are necessary for a one-semester course that is accessible to undergraduate students. Following are some of the decisions made in order to accommodate the goals of this book.

1. The book has a narrow focus as a project-oriented course on compilers. Compiler theory is kept to a minimum, but the project orientation retains the “fun” part of studying compilers.
2. The source language being compiled is relatively simple, but it is powerful enough to be interesting and challenging. It has scalar data types, strings, arrays, records (a.k.a., structs), control statements, procedures, functions, and parameters, but it relegates many other interesting language features to the project exercises. Most undergraduate students will find it challenging just to complete a compiler for the basic project language without any additional features. Graduate students will likely want to extend the basic project language with features outlined in the exercises and Appendix B.
3. The target language is assembly language for a virtual machine with a stack-based architecture, similar to but much simpler than the Java Virtual Machine (JVM). This approach greatly simplifies code generation. First, it eliminates the need to deal with general-purpose registers. And second, relative addresses for branch instructions are handled by the assembler, simplifying the amount of work that needs to be done by the compiler. Both an assembler and an emulator for the virtual machine are provided in the book’s GitHub repository.
4. No special compiler-related tools are required or used within the book. Students require access only to a Java compiler and a text editor, but most students will want to use Java with an Integrated Development Environment (IDE) such as Eclipse, NetBeans, or IntelliJ IDEA. Compiler-related tools such as scanner generators or parser generators could simplify certain tasks involved in building a compiler, but I believe that the approach used in this book makes the structure of the compiler more transparent. Students who wish to use compiler-related tools are welcome to do so, but they will need to look elsewhere to learn how to use these tools. Examples of freely available compiler-related tools include ANTLR, Coco/R, Flex/Bison, Lex/Yacc, JavaCC, Truffle, and Eclipse Xtext. In addition, while the presentation of the book uses

Java, students are free to select an alternative implementation language. Languages that support recursion and object-oriented programming will work best with the approach used in this book. Examples include Kotlin, C#, C++, Python, Scala, and Swift.

5. One very important component of a compiler is the parser, which verifies that a source program conforms to the language syntax and produces an intermediate representation of the program that is suitable for additional analysis and code generation. There are several different approaches to parsing, but in keeping with the emphasis on a one-semester course, this book covers only one approach – recursive descent parsing with several lookahead tokens. A previous edition of this book used only one lookahead token, but changes to the source language motivated the need for additional lookahead tokens. Plus, it is instructive to learn how to implement a parser using multiple lookahead tokens, especially since multiple lookahead tokens can be implemented simply and efficiently.
6. Missing from the book are a lot of favorite compiler topics such as finite automata, bottom-up parsing, attribute grammars, heap management, register allocation, and data-flow analysis. What remains fits nicely into a one-semester, project-oriented course for undergraduate students and beginning graduate students.

Why Study Compilers?

Relatively few people write commercial compilers, but I believe that the study of compiler design is an important part of a computer science education since compilers and closely related language translators are the primary tools for software development. Having a fundamental knowledge of how compilers actually work can improve one's ability to write efficient programs. In addition, learning how to write a compiler gives experience on working with a moderately large program, and it makes an excellent case study for software engineering and good program design. And the techniques used in compiler design are useful for other purposes such as writing a test harness, a support tool for process control, or an interpreter for a small, special-purpose language. One of my former students applied compiler technology to a commercial tool for testing software.

The Course Project

This book discusses implementation of a compiler for a relatively small programming language named CPRL (for Compiler Project Language), which was designed for teaching the basics of compiler construction. The target language is assembly language for CVM (CPRL Virtual Machine), a simple stack-based virtual machine. Together we will build a compiler slowly, one step at a time, with lots of template Java code in the book and even more in the book's GitHub repository to guide you through the process. Students are expected to download and study the Java code from the book repository as part of their learning experience. The end result is likely the largest single program that most undergraduate students will encounter.

Appendix A describes the decomposition of the overall project of developing a CPRL compiler into 12 smaller subprojects (numbered 0-11). Students should complete the subprojects in the specified order since each one builds on the previous.

Also, we will build the parser in three separate subprojects, focusing initially on the syntax as defined by the context-free grammar, then adding error recovery, and finally constructing abstract syntax trees for subsequent phases of constraint analysis and initial code generation. In the real world we would likely combine all three steps into one, but the division into three separate subprojects has been shown to provide a better understanding of the concepts and overall process for students new to compiler design.

Appendix B describes several extensions or variations to the basic compiler project outlined in this book. Some are more challenging than others. Ambitious undergraduate students and most graduate students will want to attempt one or more of these exercises.

Changes in the Fifth Edition

The fifth edition retains the objectives, emphasis, and general structure of the first four editions, but there are several important changes in this edition as summarized below.

1. CPRL Language Changes.

- The design and implementation of strings have been simplified, which eliminated the need for class `Padding` and provided an opportunity to add additional language features to CPRL without overly complicating the compiler project. In addition, the plus operator `+` can be used for string concatenation.
- CPRL now supports conditional expressions using the ternary operator `?:` as found in many C-based languages. So now we can write statements like the following.

```
max := x > y ? x : y;
```

An important language design issue is the precedence of this operator. In both C and Java, this operator has a very low precedence. For CPRL, rather than create a new precedence level, the conditional operator was assigned the lowest precedence that already existed in the language, the same precedence as the logical operators.

- An additional numeric type was added. Type `Byte` is an unsigned numeric type that occupies exactly 8 bits. Many programming languages have an 8-bit numeric type, although some are signed; e.g., as in Java. `Byte` values are promoted to `Integer` when used in expressions.
- The CPRL compiler now supports built-in subprograms, and three built-in functions were added to the compiler implementation.
- And finally, CPRL now supports a simple form of enumerated types (enum types). An enumerated type is defined by listing the identifiers that are the actual values of the type. Each enumerated identifier defines a constant value that is scoped to the enum type. Here is an example.

```
type Day = <Sun, Mon, Tue, Wed, Thu, Fri, Sat>;
```

A major language design decision is how the enum constants relate to integers. In some languages, they are simply integer constants. For CPRL the enum constants have some properties similar to integers, but enum types are considered to be distinct types. In particular, we can't assign enum variables or constants to integer variables, and integer variables or literals can't be assigned to variables declared as an enum type.

2. CVM Changes.

- A heap pointer, HP, was added as an internal register. The heap pointer is used only when string values are read from standard input or when strings are concatenated. Memory management is essentially nonexistent. We use a simple bump allocator that allocates heap memory when needed, and heap memory is never reclaimed; i.e., it is never deallocated.
- Five new opcodes/instructions were added to the CVM.
 1. GETBYTE: read a single binary byte from standard input
 2. GETEOF: used to detect end-of-file for standard input
 3. CHAR2INT: converts a character value to an integer
 4. INT2CHAR: converts an integer value to a character
 5. STRCAT: used for string concatenation

Ideally these new instructions would be implemented using lower-level instructions, but doing so was not possible with the existing instruction set, and adding them as machine instructions kept things simple and easy to understand. Plus, with the addition of type `Byte` to CPRL, opcodes `BYTE2INT` and `INT2BYTE` are now used by the compiler. These opcodes were defined in the previous edition of the CVM, but they were not used by the compiler implementation.

- Opcode `PUTBYTE` has been removed from the CVM. It printed was an integer-like value for the byte, and `PUTINT` works fine since byte values are promoted to `Integer` before they are printed.
- Some of the instruction byte codes changed from earlier editions, so older source code based on earlier versions of the book might need to be recompiled and reassembled in order to work properly with the current version of CVM.

3. Source Code Changes.

- Abstract syntax tree classes `SingleVarDecl` and `VariableExpr` have been removed, and all related functionality has been incorporated into classes `VarDecl` and `Variable`, respectively. The value added by the removed classes was minimal and did not fully justify their existence.

- The prefix “`edu.citadel`” was removed from all package names. For example, package `edu.citadel.assembler` is now simply package `assembler`, package `edu.citadel.common` is now package `common`, etc.
- The `main()` method for each major application of the compiler has been moved into a separate class. For example, the `main()` method for the compiler is now in class `Cpr1c` (in package `cpr1c`).

In addition, there are many new paragraphs, various rewordings to make the presentation easier to understand, and a few corrections. In developing this new edition, several other possible changes to CPRL were considered, implemented, and then rejected to keep the book at a reasonable length and level of complexity. Overall, I am extremely pleased with the resulting new edition of this book.

Structure of the Book

The book chapters are organized in a natural progression from general concepts to step-by-step details for implementing the compiler project. Each chapter builds on the previous chapters, and forward references have been kept to a minimum. A few sections can be omitted or postponed without loss of continuity, but in general, the book is designed to be studied from front to back.

The most difficult chapters are 3, 6, 8, and 13, so don't try to rush through those chapters. Not coincidentally, these are also the longest chapters. The good news is that the last four chapters covering arrays, enumerated types, strings, and records are relatively straightforward, assuming mastery of the preceding material. These last four chapters can be covered relatively quickly, and it is not uncommon for the strongest students to work ahead at this point and to finish related compiler subprojects early.

Chapter 1 introduces basic concepts and terminology in the study of compilers. Following the book by David A. Watt and Deryck F. Brown [Watt 2000], Chapter 1 uses tombstone diagrams as a visual aid to explain many of the concepts. Sections 1.2, 1.3, and 1.6 are required. The other sections can be omitted or postponed without impacting the material in subsequent chapters, but doing so is not recommended.

Chapter 2 presents an overview of the structure of a compiler. In a sense, it sets the context for the remaining chapters. It can be omitted or postponed, but I always spend one class period on this chapter.

Chapter 3 provides an overview of how programming languages are defined, with detailed coverage of context-free grammars. The complete context-free grammar for CPRL, the source language for the compiler project, is defined in Appendix D.

Chapter 4 gives a brief overview of CPRL. A more complete definition of CPRL is provided in Appendices C and D.

Chapter 5 presents the details of implementing a scanner for CPRL, including an overview of several related classes found in the book's GitHub repository.

Chapter 6 is the longest and one of the most complicated chapters in the book. It includes a discussion of grammar analysis to compute first sets and follow sets plus details on the

implementation of a recursive descent parser for CPRL. As defined in the compiler project, the initial version of the parser concentrates primarily on verification that a source program conforms to the syntax defined by the context-free grammar for CPRL.

Chapter 7 extends the parser developed in Chapter 6 to perform error recovery so that multiple errors can be detected and reported by the compiler.

Chapter 8 contains a detailed discussion of abstract syntax trees. Abstract syntax trees provide an intermediate representation for source programs that can be used for additional analysis and code generation. In this chapter, the parsing methods are modified to construct an abstract syntax tree for the source program.

Chapter 9 uses the abstract syntax trees from Chapter 8 to perform constraint analysis, with emphasis on checking conformance to CPRL's type rules as defined for CPRL/ $\emptyset$, a major subset of CPRL.

Chapter 10 gives a brief overview of the CVM, the virtual machine that serves as the target for CPRL programs. The CVM has a stack architecture that is similar to but much simpler than the Java Virtual Machine (JVM). Appendix E contains additional information about the CVM, including complete definitions for every CVM instruction.

Chapter 11 modifies the abstract syntax trees to generate assembly language for CPRL/ $\emptyset$. The GitHub repository provides an assembler that can be used to generate the actual CVM machine code.

Chapter 12 presents an overview of code optimization. The assembler provided on the course web site implements several optimizations, but none of the compiler projects require mastery of the material from this chapter. Therefore, this chapter can be postponed if desired, but it should not be omitted.

Chapter 13 extends constraint analysis and code generation to handle subprograms. It includes additional coverage of scope and a detailed discussion of calling conventions and activation records. Similar to Chapter 6, this chapter is long and can be quite complicated for many undergraduate students. But mastery of this material is essential for a course on compilers, even if it means that less time is available for the last four chapters.

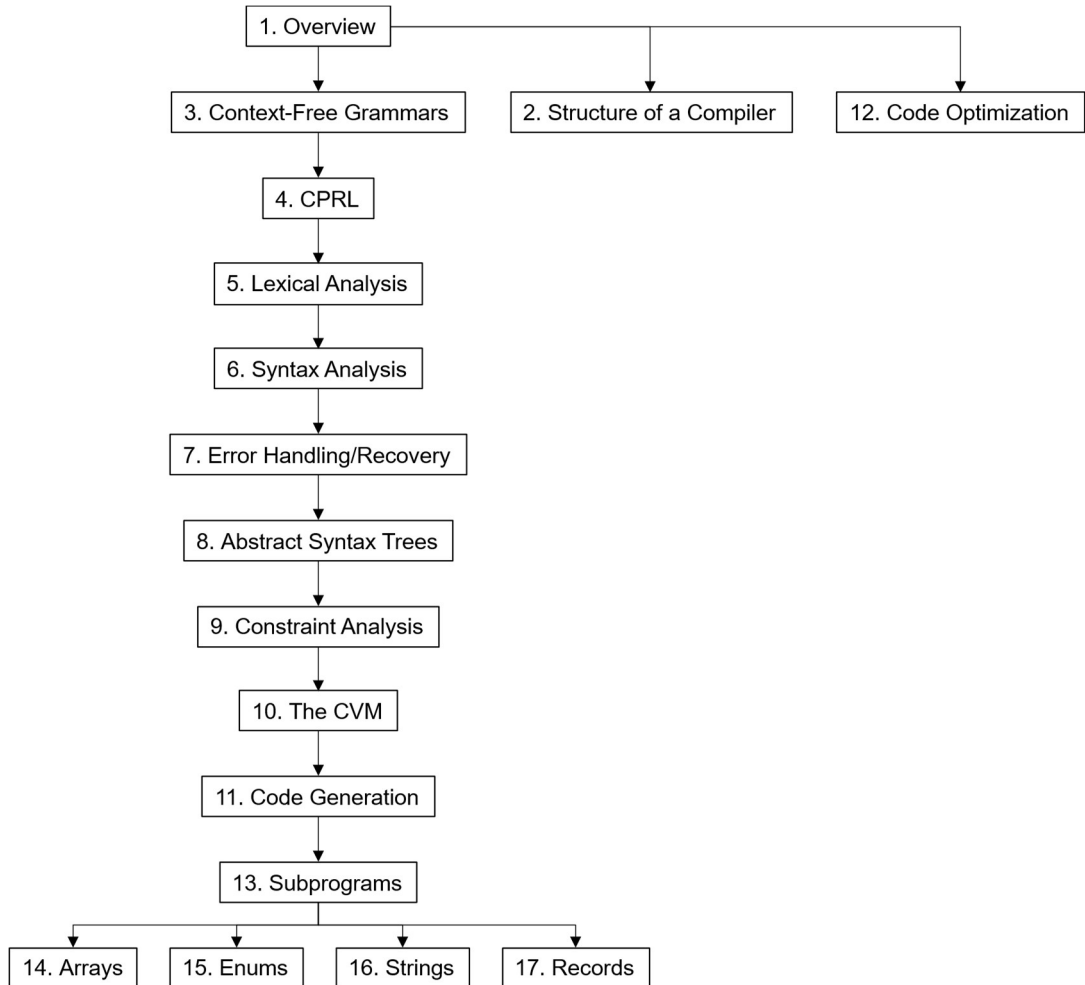
Chapters 14-17 extend constraint analysis and code generation for arrays, enumerated types, strings, and records, respectively. Projects corresponding to these chapters are somewhat simpler than most of the preceding projects and can be completed relatively quickly. Implementing records completes the compiler project, but students are always encouraged to attempt some of variations and extensions outlined in Appendix B.

Chapter Dependencies

As illustrated in the diagram on the next page, Chapters 2 and 12 can be covered any time after Chapter 1, and Chapters 14-17 can be covered any time after Chapter 13. Otherwise, most chapters should be covered in the order presented in the book. The recommended approach is to cover the chapters in order 1-17 using the appendices as supplementary material. Many of the exercises in the last four chapters assume that compiler subprojects

for previous chapters have been implemented. For example, some exercises in Chapter 17 use concepts from Chapters 14-16.

Chapter Dependencies



Book Resources

The repository for this book at <https://github.com/SoftMoore/CPRL-Java-5th> contains a number of related resources as follows.

- Java source code that implements the CVM, the target machine for the compiler project.
- Java source code that implements an assembler for the CVM. The compiler project targets assembly language for the CVM rather than actual virtual machine bytecode.
- Java source code for a disassembler; i.e., a program that takes CVM machine code and converts it back into assembly language. This program can be useful for debugging the compiler projects and for understanding how machine code is laid out in memory.
- Java source code or skeletal Java source code for many of the classes described in the book so that students don't need to start from scratch to create their compilers. Some of these classes suggest implementation ideas for other classes whose implementations are either not provided or are only partially provided. For example, source code for a procedure declaration might be relevant when implementing a function declaration.
Students should begin each phase of their compiler project by trying to understand the related Java source code that is provided.
- Many examples of correct and incorrect CPRL programs that can be used to test a compiler. The final compiler should reject all incorrect CPRL programs with appropriate error messages, and it should generate semantically equivalent assembly language for all correct CPRL programs. Using the provided assembler and CVM emulator, all correct CPRL programs can be run to compare observed and expected results. Students are strongly encouraged to develop additional test programs.
- Sample Windows command scripts and Bash shell scripts for running and testing various stages of the compiler. For example, rather than trying to compile and test programs one at a time, there are command/shell scripts for running the compiler on all CPRL source files in the current working directory. These scripts are useful for testing the compiler against collections of correct and incorrect CPRL programs.

Appendix A provides additional details about the resources available on the book repository and how those resources fit into the overall structure of the course compiler project.

Java and Object-Orientation

As indicated in the book's title, the language used herein for implementing a compiler is Java, but other options are certainly possible. In fact, one of the project exercises listed in Appendix B is to write the compiler in a different language, and some of my former students have done this.

Since Java is an object-oriented language, every component of the compiler is structured in terms of classes and objects – the scanner is an object, the parser is an object, tokens are objects, etc. But the full power of “object-orientation” doesn't come into play until Chapter

8, where the details of abstract syntax trees are introduced. The approach used in this book for implementing abstract syntax trees fully exploits Java’s object-oriented facilities for inheritance and polymorphism. As an example, consider that code generation for a list of statements looks like the following.

```
for (var stmt : statements)
    stmt.emit();
```

The list could include `if` statements, assignment statements, loop statements, etc., and therefore different statements generate code in different ways. But at the level of programming abstraction shown in this example, we only need to know that each statement has an `emit()` method that writes out the appropriate object code.

Similarly, the code to check constraints for a list of CPRL subprogram declarations looks like the following.

```
for (var decl : subprogDecls)
    decl.checkConstraints();
```

Note that the subprogram declarations in the list could be either procedure declarations or function declarations.

Which Version of Java?

The exposition in this book and the resources provided in the book’s GitHub repository assume Java version 17 or later. In particular, interfaces `VariableDecl` and `Initializer` are declared to be sealed interfaces, and several classes make use of pattern matching for the `instanceof` operator.

The Java source code also makes use of the arrow syntax for switch statements and switch expressions, and Java source files frequently use local variable type inference with the keyword `var`, especially in conjunction with the `new` operator. Here is an example.

```
ArrayList<Statement> statements = new ArrayList<>(20);    // older style
var statements = new ArrayList<Statement>(20);           // using var
```

Formatting Source Code

Every programmer has his/her preferred way to format and indent source code. Should curly braces be aligned? Should code be indented 2, 3, or 4 spaces? What is the best way to format an `if` statement or a `while` loop? Sometimes arguments about how source code should be formatted seem to take on religious-like fervor. I avoid such arguments, for I have found the **one true way** that Java source code should be formatted. ☺ But if you are a nonbeliever, please feel free to reformat the Java source code according to your project standards or personal tastes. (FYI: The formatting style used in this book is loosely based on the GNU style, which is not very popular but which I have personally found to be the most readable. As the saying goes, “Beauty is in the eye of the beholder”.)

Acknowledgements

When I was first learning about computers as a young man, I was in awe of compilers. That awe evolved into curiosity, which led to a lot of thought and study about how compilers are constructed, which led to a lot of contemplation about how best to teach an introductory course in compilers, which eventually led to my writing this book. Along the way I was guided and influenced by a number of teachers, colleagues, students, and books. I would like to acknowledge those who have been the most influential on my thinking.

I was first introduced to compilers many years ago when I “unofficially” audited a course on compiler design given by Richard LeBlanc at Georgia Institute of Technology (a.k.a. Georgia Tech). For reasons I no longer remember, I was unable to take the course for credit. But auditing it, especially under Richard LeBlanc, was enough to motivate me to learn more. I was fortunate enough to have Richard LeBlanc later on for another course on programming language design.

My next introduction to compilers came in a professional development course given by Frank DeRemer and Tom Pennello, with a guest lecture or two by Bill McKeeman. They will not remember me, but I am grateful to have learned more about compilers from teachers and researchers of their caliber.

I have also been inspired by several compiler books, especially two of them that took a pedagogical approach similar to this one. The first book is *Brinch Hansen on Pascal Compilers* by Per Brinch Hansen (Prentice Hall, 1985). That book is a little out of date now, but it had one of the most readable descriptions of compilers when it was first released. A second, more modern book is *Programming Language Processors in Java: Compilers and Interpreters* by David A. Watt and Deryck F. Brown (Prentice Hall, 2000). I follow their treatment of tombstone diagrams when explaining compilers and interpreters in Chapter 1. Many years ago I used the Brinch Hansen book as a textbook in my compiler courses, and I used the Watt-Brown book several times when it was first published.

It is important to acknowledge former students at Johns Hopkins University, the College of Charleston, and The Citadel, and to thank these institutions for allowing me to explore my ideas about writing compilers. I am particularly grateful to the following students who had the most influence on my thinking about how to teach compilers: Rob Ring, Scott Stanchfield, Logan Daigle, Zack Aardahl, Ben Hunter, Dalton Hazelwood, Gordon Finlay, Josh Terry, Davis Jeffords, Mike Dalpee, Matthew Blair, Mafer Contreras, Jared Johnson, Robert Roser, Shiloh Smiles, Noah Klepper, Robert Powell, Eddie Portelles, and Luke Tramontozzi. Some of the earlier students at Johns Hopkins suffered through my courses as I was trying to crystallize my approach to teaching compilers, and I am grateful for their feedback. They would barely recognize my course if they took it today.

In addition, I want to acknowledge Vince Sigillito, who served as Chair of the Computer Science program at the Johns Hopkins Part-Time Programs in Engineering and Applied Science, for first allowing me to teach a course on compilers many years ago. I remember telling Vince that I wanted to teach a course on compilers so that I could learn more about them. He had no objections and even indicated that he had done something similar in the past. Compiler Design has evolved into my favorite course.

I would also like to acknowledge the following individuals who provided invaluable advice and feedback on compilers in general and on the presentation and exposition in this book: Dr. Richard J. LeBlanc, Jr., Professor Emeritus, Georgia Institute of Technology and Professor Emeritus, Seattle University; Dr. Arthur B. (Art) Pyster, retired, former Professor of Systems Engineering, former Associate Dean for Research, George Mason University; Dr. Shankar M. Banik, Professor and Head, Department of Cyber and Computer Sciences, and Director of The Citadel DoD Cyber Institute, The Citadel; and Dr. George L. Rudolph, Professor and Chair, Department of Computer Science, Utah Valley University. I am most appreciative and humbled that they spent part of their valuable time assisting me with this effort.

A special acknowledgement goes to Dr. David Buzatto, Professor de Informática, Federal Institute of Education, Science and Technology of São Paulo, Brazil. Dr. Buzatto performed a thorough review of this edition of the book, uncovering many issues and providing numerous suggestions for improvement. I am ashamed to say that some of the issues he discovered have existed for several earlier editions. Thank you, Dr. Buzatto. I am eternally grateful for your assistance.

Finally, I would like to acknowledge Kayran Cox Moore, my wife of many years, for proofreading and providing invaluable feedback on several drafts of this book. She might not understand compilers or Java programming, but she has a deep understanding of English grammar and sentence structure, and she has no reservations about correcting my errors or improving my writing. Any grammatical errors remaining in this book are a result of my stubborn refusal to follow her advice, or, more likely, they were introduced after she proofread that part of the book. I also want to thank Kayran for being my anchor in life and the source for most of what is good about myself.